

Web Application Penetration Testing Report

Mediroza General Hospital

Prepared By: Joel Biju

Internship: NETWORKWALKS Cybersecurity & Ethical Hacking (Batch B083)

Mentor: Waqas Karim (CCIE)

September 2026

Document Control

Parameter	Detail
Assessment Type	Black-Box Web Application Security Assessment
Report Version	1.0
Report Date	September 2026
Target URL	https://medirozahospital.com
Authorisation	Written authorisation provided prior to testing
Duration	3 Days (September 2026)

1 Executive Summary

Mediroza General Hospital engaged Networkwalks to perform a full black-box penetration test against their public-facing web infrastructure. The assessment was carried out by Joel Biju (Batch B083) over a three-day period in September 2026, with written authorisation granted by the client prior to any testing activity[cite: 2].

The assessment revealed a deeply concerning security posture. The web application suffered from multiple critical vulnerabilities that, when chained together, allowed the assessor to bypass authentication entirely, access confidential patient medical records, crack encrypted files, and ultimately retrieve a publicly exposed database backup containing the personal details, salaries, and shareholding information of every employee at the hospital. None of these actions required advanced tooling — they were achieved using standard, widely-available security tools and publicly documented techniques.

1.1 Summary of Findings

#	Finding	Severity
1	SQL Injection — Authentication Bypass on Patient Portal	CRITICAL
2	Exposed Database Backup File (Staff Salaries & Shareholder Details)	CRITICAL
3	Weak / Easily Cracked PDF Encryption Passwords	HIGH
4	Sensitive Path Disclosure via robots.txt	MEDIUM
5	Directory Listing Enabled on Multiple Directories	MEDIUM
6	Username Enumeration on Login Form	MEDIUM
7	Verbose SQL Error Messages Exposed to End User	LOW

2 Scope and Methodology

2.1 Scope

Parameter	Details
Target URL	https://medirozahospital.com
Assessment Type	Black-Box Penetration Test
Scope	Target domain only (medirozahospital.com)
Rules of Engagement	No social engineering. No Denial of Service. No testing outside agreed scope.
Authorisation	Written authorisation provided by the client prior to testing.
Duration	3 Days (September 2026)

2.2 Methodology

The assessment followed a structured methodology aligned with the OWASP Testing Guide and industry best practice[cite: 3]. Testing was performed in a systematic, non-destructive manner, progressing through the following phases:

Phase 1 - Passive Reconnaissance: Collected publicly available information about the target using WHOIS lookups, DNS enumeration, IP ownership analysis, and search engine dorking[cite: 3]. No direct requests were made to the target during this phase.

Phase 2 - Active Reconnaissance: Performed active scanning against the target including port scanning (`nmap`), WAF detection (`wafw00f`), technology fingerprinting (`WhatWeb`), and directory/file enumeration (`Gobuster`)[cite: 3]. The `robots.txt` and `sitemap.xml` files were reviewed for path disclosure.

Phase 3 - Vulnerability Identification: Manually analysed the web application’s authentication mechanisms, input handling, directory structure, and file exposure[cite: 3]. Identified SQL Injection, username enumeration, and directory listing vulnerabilities.

Phase 4 - Exploitation: Exploited identified vulnerabilities to demonstrate real-world impact. Successfully bypassed authentication via SQL Injection, accessed restricted patient records, cracked encrypted PDF files, and discovered a publicly exposed database backup containing highly sensitive HR data.

Phase 5 - Reporting: Documented all findings with evidence, assessed risk ratings using CVSS scoring, and provided actionable remediation recommendations.

2.3 Tools Used

Tool	Purpose
<code>nmap</code>	Port scanning and service version detection
<code>wafw00f</code>	Web Application Firewall detection
<code>WhatWeb</code>	Technology fingerprinting
<code>Gobuster</code>	Directory and file brute-forcing
<code>Burp Suite</code>	HTTP traffic interception and analysis
<code>curl</code>	HTTP header and response inspection
<code>pdfcrack</code>	PDF password recovery
<code>John the Ripper</code>	Offline hash cracking
<code>ExifTool</code>	File metadata analysis
<code>Browser</code>	Manual web application exploration

3 Attack Walkthrough

The following section documents the complete attack chain from initial passive reconnaissance through to the extraction of the hospital’s most sensitive internal records.

3.1 Milestone 1 — Initial Access: Patient Portal Breach

Step 1: Passive Reconnaissance

The assessment began with WHOIS enumeration to identify the registrar and hosting provider. The domain was registered via Namecheap (created August 2026) and hosted on Namecheap’s shared hosting infrastructure at IP address 199.188.201.16, confirmed as part of Namecheap’s allocated range (199.188.200.0/21). The specific server was identified as `server274.web-hosting.com` (shared #1) — a cPanel-based shared hosting environment.

Step 2: WAF Detection & Port Scanning

WAF detection using `wafw00f` confirmed the site was protected by a LiteSpeed WAF. Nmap port scanning revealed the server exposed standard web ports (80, 443) along with mail-related ports (110, 143, 465, 587, 993, 995) and FTP (21) — consistent with a cPanel shared hosting environment[cite: 6]. No cPanel management ports (2082/2083) were directly exposed.

Step 3: Sensitive Path Disclosure via robots.txt

Reviewing the site's `robots.txt` file intended to tell search engines which pages to avoid indexing inadvertently revealed three sensitive directories that the development team had attempted to hide: `/patient/`, `/staff/`, and `/old/`. The `sitemap.xml` additionally confirmed only four public-facing pages existed, meaning these three directories were deliberately excluded from public view — yet were fully accessible.

Step 4: Directory Listing Patient Portal Structure Exposed

Navigating directly to `https://medirozahospital.com/patient/` revealed that directory listing was enabled — a serious misconfiguration that displays the full contents of a web server directory to any visitor. The exposed files included:

File	Significance
<code>reports/</code>	Directory likely containing the patient lab report PDFs
<code>login.php</code>	Patient portal authentication page — the entry point for exploitation
<code>portal.php</code>	Main patient portal dashboard — accessible after login
<code>download.php</code>	File download handler — used to serve PDFs to authenticated users
<code>error_log</code>	Server error log — can expose file paths, credentials, and stack traces
<code>logout.php</code>	Session termination handler

Step 5: SQL Injection Authentication Bypass

The patient portal login form at `/patient/login.php` was found to be vulnerable to SQL Injection[cite: 9]. The application also exhibited username enumeration as it returned different error messages depending on whether the username existed ("Invalid username" vs "Invalid password").

The payload used to bypass authentication:

- **Username field:** `' OR 1=1 --`
- **Password field:** `anything`

This payload closes the SQL string prematurely (`'`), appends a condition that is always true (`OR 1=1`), and comments out the remainder of the query (`--`).

3.2 Milestone 2 — Cryptographic Analysis: PDF Password Recovery

The three downloaded patient pathology reports were password-protected using PDF Standard V2.3 encryption (RC4, 128-bit). The tool `pdfcrack` was used with the `rockyou.txt` wordlist. All three PDF passwords were recovered successfully:

File	Patient	Password Status / Recovered
<code>patient_report_1.pdf</code>	Sipho Dlamini	Recovered
<code>patient_report_2.pdf</code>	Priya Reddy	password
<code>patient_report_3.pdf</code>	Emily Thompson	!@#%\$^&*

3.3 Milestone 3 — Critical Data Exposure: Database Backup Discovered

The `/old/` directory — disclosed by the `robots.txt` file — contained `mediroza_db_backup2019.sql`[cite : 15]. The backup file contained two tables with sensitive information[cite : 15] :

Staff Table (30 Employees)

Full name, job title, department, email address, phone number, national ID number, monthly salary (ZAR), and date joined[cite: 15]. Examples include[cite: 15]:

- Dr. Johan van der Merwe (Medical Director) | R 160,000[cite: 15]
- Sarah Botha (Chief Financial Officer) | R 152,000[cite: 15]
- Dr. Rajesh Naidoo (Chief Pathologist) | R 138,000[cite: 15]
- Dr. Vikram Chetty (Anaesthetist) | R 135,000[cite: 15]
- Linda Fourie (Receptionist) | R 19,000[cite: 15]
- Andile Mbeki (Ward Clerk) | R 21,000[cite: 15]

Shareholders Table (10 Shareholders)

Shareholder	Share %	Shares Held	Class
Dr. Rajesh Naidoo	18.0%	180,000	Ordinary
Cedar Health Holdings (Pty) Ltd	15.0%	150,000	Ordinary
Dr. Johan van der Merwe	12.0%	120,000	Ordinary
Reddy Family Trust	11.0%	110,000	Ordinary
Thabo Molefe	10.0%	100,000	Ordinary
Sarah Botha	9.0%	90,000	Ordinary
Dr. Ahmed Kara	8.0%	80,000	Preferential
Naledi Zulu	7.0%	70,000	Ordinary
Michael Roberts	6.0%	60,000	Ordinary
Dr. Vikram Chetty	4.0%	40,000	Preferential

4 Technical Findings Details

4.1 Finding 1: SQL Injection Authentication Bypass (CRITICAL)

CVSS Score: 9.8 | CWE: CWE-89 | Component: Patient Portal Login (/patient/login.php)[cite: 18]

Description

The patient portal login form passed user input directly into a raw MySQL query with no input validation or prepared statements[cite: 18].

Remediation

Immediately replace all raw SQL queries in the application with parameterised queries (prepared statements)[cite: 18].

4.2 Finding 2: Exposed Database Backup File (CRITICAL)

CVSS Score: 9.1 | CWE: CWE-538 | Component: Web Server (/old/mediroza_db_backup_2019.sql)[cite: 20]

Description

A complete SQL database backup file was publicly accessible without authentication[cite: 20].

Remediation

Remove the backup file from the web server immediately[cite: 20]. Store database backups in a secure, non-web-accessible location[cite: 20].

4.3 Finding 3: Weak PDF Encryption Passwords (HIGH)

CVSS Score: 7.5 | CWE: CWE-521 | Component: Patient Lab Reports[cite: 21]

Description

Patient pathology reports used weak passwords that appeared in standard wordlists[cite: 21].

Remediation

Enforce a strong password policy for all files shared with patients (minimum 12 characters with mixed types)[cite: 17, 21].

4.4 Finding 4: Sensitive Path Disclosure via robots.txt (MEDIUM)

CVSS Score: 5.3 | CWE: CWE-538 | Component: robots.txt[cite: 22]

Description

The robots.txt file disclosed sensitive paths (/patient/, /staff/, /old/)[cite: 22].

Remediation

Remove sensitive internal paths from robots.txt and rely on proper server-side access controls[cite: 22].

4.5 Finding 5: Directory Listing Enabled (MEDIUM)

CVSS Score: 5.3 | CWE: CWE-548 | Component: Web Server (/patient/, /staff/, /old/)[cite: 23]

Description

Directory listing enabled users to view all files within server directories[cite: 23, 24].

Remediation

Disable directory listing in the web server configuration (e.g., Options -Indexes)[cite: 17, 24].

4.6 Finding 6: Username Enumeration on Login Form (MEDIUM)

CVSS Score: 5.3 | CWE: CWE-204 | Component: Patient Portal Login[cite: 26]

Description

Different error messages were returned for invalid usernames vs incorrect passwords[cite: 26].

Remediation

Return generic error messages for all authentication failures ("Invalid username or password")[cite: 17, 26].

4.7 Finding 7: Verbose SQL Error Messages Exposed (LOW)

CVSS Score: 3.1 | CWE: CWE-209 | Component: Patient Portal Login[cite: 26]

Description

Raw MySQL error messages were displayed to end users when malformed queries were executed[cite: 26].

Remediation

Disable display of PHP/MySQL errors in production (display_errors = Off)[cite: 17, 27].

5 Remediation Summary

Immediate Actions (Within 24–48 Hours)

- Take the patient portal offline or disable login until SQL Injection is patched[cite: 17].
- Remove mediroza_db_backup_2019.sql from the web server[cite: 17].
- Disable verbose PHP/MySQL error reporting (display_errors = Off)[cite: 17].

Short-Term Actions (Within 1–2 Weeks)

- Enforce strong password policies for encrypted files[cite: 17].
- Sanitise robots.txt and disable directory listing globally (Options -Indexes)[cite: 17].
- Implement generic authentication error messages[cite: 17].

Medium-Term Actions (Within 1 Month)

- Implement a Web Application Firewall (WAF)[cite: 17].
- Conduct a full audit of all hosted server files[cite: 17].
- Implement multi-factor authentication (MFA)[cite: 17].

Long-Term Actions (Ongoing)

- Perform regular penetration testing[cite: 17].
- Adopt a Secure Development Lifecycle (SDLC)[cite: 17].
- Provide security awareness training to staff[cite: 17].

A Finding Severity Definitions

Rating	CVSS Score	Definition
CRITICAL	9.0–10.0	Exploitation will cause severe harm. Immediate resolution required.
HIGH	7.0–8.9	Exploitation will cause significant harm. Short-term resolution required.
MEDIUM	4.0–6.9	Exploitation has a meaningful impact. Address during routine maintenance.
LOW	1.0–3.9	Exploitation causes minimal impact.
INFO	0.0–0.9	Informational finding. Areas for general improvement.